TRACE-Eval: change-aware, cost-bounded regression test selection for LLM agent systems

Ciprian Păduraru
University of Bucharest
ciprian.paduraru@unibuc.ro

Abstract

An LLM agent system is assembled from skills, prompts, tools, harness code and configuration, and verified by a suite T of tests whose cost $c(t)$ is dominated by tokens. After a change Δ, a team either reruns all of T or guesses. We formulate selection, prioritization and stopping as one problem: choose tests from T, one at a time, under a budget B, so that the regressions Δ introduces are found with high probability at low cost. The change becomes a set of components $C(\Delta)$ (code lines and functions, skill procedures and prose, agent fields, configuration keys), each broken with a prior probability π_c. A test's sensitivity $s_{tc} = P(t \text{ fails} \mid c \text{ broken})$ is calibrated from the dependencies observed in execution traces and from the record of earlier changes, and a test that already fails on the unchanged system, with rate f_t, is discounted. The posterior over which subset of the components is broken is kept exactly; the next test maximizes the expected information gain per token; the run stops with REGRESSION at the first failure of a test that used to pass, with CLEAN when $P(\text{some component broken} \mid \text{outcomes}) < \varepsilon$, and never before every mandatory property that Δ may affect has had a test. We evaluate on a testbed: a multi-agent content-operations application whose agents are played by an open model (Gemma 4 E4B), 180 tests, 43 controlled changes applied as file edits, and a ground truth measured per model by majority over five runs. Dependencies the test authors declared by hand miss 38% of the regressions; dependencies observed in traces find 86% within 20% of the full-suite tokens; two published methods reproduced on the testbed, CASPER's slices and a risk-aware bandit, find 59% and 57% because neither reads the change; the proposed method finds 78% for 3% of the tokens, 84% of the changes that break a safety property, and gives no false verdict on the 14 changes no test reveals.

1 What a team needs

1.1 The setting

A platform such as the agent layer of Adobe Experience Manager (AEM) ships dozens of agents. Each is a bundle of a system prompt, a list of tools, a list of skills (prompt files with a procedure the model follows), and settings, running on a shared harness that dispatches tool calls, retries, enforces approvals and delegates between agents. One prompt may cross several agents, through a plan the orchestrator writes (Section 2). The platform is verified by scenario tests, which run a request end to end and check the final state and the trace, and by datasets of prompts with expected routings or tool calls. Every test is a run of a model. It costs tokens and seconds, and the cost varies by a factor of ten to a hundred between a one-shot routing item and a workflow across five agents.

After a change, the question is the one regression testing has always asked [33]: which tests to run (selection), in what order (prioritization), and when to stop. Three things make the classic answers [24, 25] hard to transfer:

- much of the "code" is natural language, so coverage of a skill or a prompt is not a line of a control-flow graph;

- the outcome of a test is the behaviour of a model, so it is not a function of the code alone and can vary from run to run;
- the cost of a test is not one unit, so an order that finds a fault after three tests can still have spent a third of the budget.

1.2 The desired features

The AEM team, asked what they lack and want in a prototype, listed the following. Each is built into the testbed; the section in parentheses shows it.

- **Selection, prioritization and stopping under a budget**, with a verdict a developer can act on (Section 4).
- **A catalog of properties**, the behavioural requirements the tests check, drafted by a model and edited by people (Section 2).
- **Coverage of each change**: the share of what it touches that the selected tests exercised (Section 3).
- **Per-test metadata**, declared by the author and observed in traces (Section 3).
- **Trace assertions** on the final state, the tools and the order of agents (Section 2).
- **Workflows** across several agents as a unit of testing and of coverage (Section 2).
- **Environment failures kept apart** from the agent's failures (Section 3).
- **OpenTelemetry traces** with the plan, the routing and the approvals as events [21] (Section 2).

1.3 What exists, and what is missing

Table 1 places this paper in the literature. The two closest methods, CASPER and risk-aware adaptive evaluation, do not read the diff. This paper's claim is that the diff, mapped to the tests through execution traces, is worth a lot, and that a sequential Bayesian treatment of the outcomes turns selection, prioritization and stopping into one decision.

The contributions are:

1. a formulation of change-aware, cost-bounded testing of agent systems, with the metrics of regression testing adapted to token costs (Section 3);
2. a method that extracts a change set at the granularity of code lines, skill procedures, agent fields and configuration keys, maps it to tests through dependencies observed in traces, ranks tests by expected information per token in a Bayesian model calibrated from history, and stops with a verdict (Section 4);
3. a testbed: a multi-agent application, 180 tests, 43 controlled changes and a mutation-based ground truth derived per model, on which strategies are compared by offline replay (Sections 2 and 5);
4. an evaluation against six baseline strategies and two published methods reproduced on the testbed (Section 5).

2 How we built the testbed

The application. We wanted an application small enough to read in an afternoon and shaped like the platforms the question comes from. ContentOps (Figure 1) is a content-operations platform in the style of a web content manager: pages with versions, assets, publishing with reviews and schedules, link validation, SEO metadata, translation, governance with consent and approval tokens, rollback, brand rules, analytics and a site audit. An orchestrator and ten specialists, three of them (brand governance, data insights, site optimization) modelled on Adobe's, run on a shared harness. Each specialist follows a skill: a Markdown file with prose rules and a machine-readable procedure that the model reads as numbered steps, with the skills

Table 1: Related work and why it does not transfer as it is.

line of work	what it does	here
regression test selection and prioritization [6, 7, 16, 24, 25, 33]	safe selection on control-flow graphs; orders scored by APFD and its cost-cognizant form	much of the "code" is prose and the outcome is a model's; the metrics carry over (Section 3)
information retrieval over the diff [26]	ranks tests by text similarity to the change	reproduced as `ir_tfidf`
history and time windows in continuous integration [8, 17]; learned selection [14, 28]; black-box diversity [22]	ranks by past failures, a learned model or the diversity of the tests	assume unit costs and deterministic outcomes; the history factor returns in `cost_aware`
regression testing of LLM applications [13]	names the obstacles: no crisp oracle, brittle prompts, nondeterminism	a diagnosis, not a selector
efficient evaluation of language models [10, 20, 23, 31]	picks the subset of a benchmark that reproduces a score	selects once, for all models, not after a change
GBV-PD [4]	ranks tests from a parameter update	needs gradients the models here do not expose
fault injection for agents [27, 29]	perturbs the runtime	the testbed mutates the artefacts under change instead
CASPER [19]	slices the suite by behaviour, probes representatives of each slice on the new version, ranks the slices by drift	change-aware only through drift; reproduced as `casper_slices`
risk-aware adaptive evaluation [15]	a bandit allocates runs by impact and by the outcomes seen so far	does not read the diff; reproduced as `risk_aware_ts`

it includes written out and every setting shown with its value. Thirteen skills, 32 tools over deterministic in-memory services and one configuration file make up the rest. The orchestrator delegates a request to one specialist or, when it needs several, writes a plan, hands each step its own instruction and the ids earlier steps reported, and asks a person before any step that publishes, schedules or takes pages down; the approver's answer is fixture data, so a run is repeatable.

The structure is deliberate: components with a wide blast radius (the shared skills, the permission check most specialists may call, the retry helper around every tool call) beside narrow ones (the approval gate matters to one test), and rules that live only in a skill (publish only with permission) beside rules a tool or a service also enforces (approval tokens, roles). That is what makes selection hard, and what lets us discuss defence in depth: when a change removes a check from a skill, usually nothing below the skill catches the mistake.

The tests. We wrote the tests the way a team would write them for the application, before and independently of the changes (Table 2). Every test names its risk level, the properties it checks, and the components its author declares it depends on. We check facts, not wording: a check that the answer contains "broken" would fail a model that says "invalid". Two cases with a rubric also have their answer judged by a model, after the deterministic checks pass. The declared block is incomplete on purpose, as it is in practice: nobody declares the shared skill reached through an include, the harness code or the configuration keys.

The properties. The AEM team wanted "prompt, properties and threshold", with categories. Table 3 is the catalog; each property also has a description and the components that state it, its sources, and the mandatory ones are release blockers. Each test lists the properties it checks. A model can draft the same catalog from the skills' rules, the agent definitions and the configuration

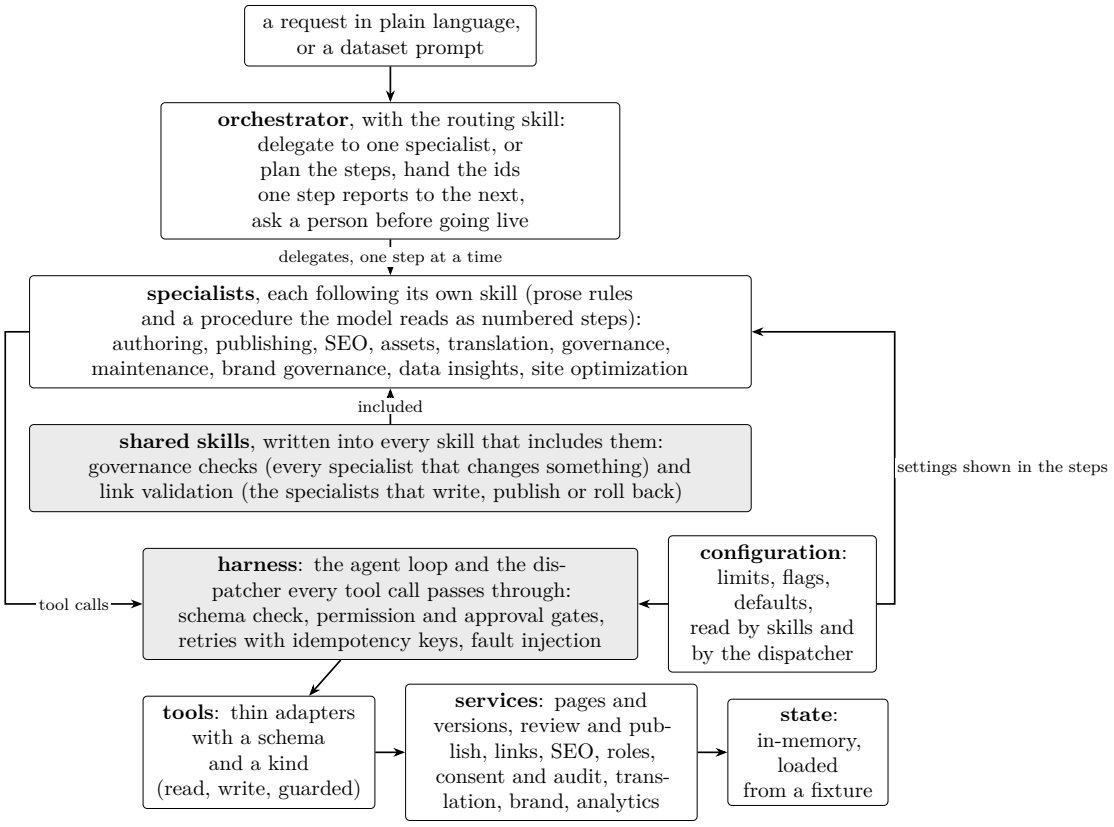


Figure 1: ContentOps, one request from top to bottom. Grey: shared by many agents, so a change there has a wide blast radius.

guards, and tag the tests, for a person to edit: its catalog is good and traces every property to the component that states it; its tags of the tests need the person's review.

The environment. An open model plays the agents: Gemma 4 E4B, served with vLLM and decoding greedily, so that the same prompt gives the same answer up to the arithmetic of batching. Qwen3.8-27B and gpt-oss-120b are second and third models, used as they are served, without sampling. Every run is traced (Table 4) and exports as OpenTelemetry spans. A run the environment broke, because the server or the network did not answer, is marked and left out, as if it had not been made. The whole suite runs $K = 5$ times on the current version and on every changed copy; the strategies are compared afterwards by replaying the recorded outcomes (Section 5).

The changes. We wrote 43 controlled changes as file edits, in the spirit of mutation analysis [5, 9], each to resemble a regression a team would ship (Table 5). Two rules shaped the set:

- a change meant to break something touches an input the model acts on; a keyword weight or the order of an agent's intents is not one;
- a removed step goes from both the procedure and the prose of a skill, or the model does the step anyway.

We left out what told nothing about selection: twins of other changes, crashes, and a change that broke most of the suite. Ten of the changes aim at the workflows. Which test reveals which change was measured afterwards, never designed, and no strategy is told.

Table 2: The suite of 180 tests.

kind	count	gives	checks
scenario test	91	an entry agent; a request in plain language ("Please publish the welcome page"); a context (the user the agent acts for, the ids the request is about, any approval token); a fixture (the platform's starting data); a fault profile (healthy, one transient failure of a tool, one lost acknowledgement, or a tool permanently down)	the final state (the page is published, the audit log has one entry); the tools required and forbidden; attempt counts; facts the answer must report (a job id, which link is broken); a step limit
workflow test	15	the same, for one of six multi-agent workflows: a campaign launch across five specialists, a site optimization across four, a legal takedown, a localized price update, the rollback of a broken release, a privacy request; each with a variant where a step must stop the plan	the same, and the order of agents and of tools
routing item	50	a prompt	the expected delegation
tool-call item	24	a prompt	the expected tool call

Table 3: The catalog of properties: the threshold is on the pass rate of a property's tests.

category	properties	checked by	threshold	mandatory
correctness	task completion; correct routing; tool trajectory	state; trace, dataset	0.95	no
safety	authorization; guarded mutation; no broken publish; privacy; human approval; brand compliance	state, trace	1.0	yes
resilience	retry; idempotency; fail safe	state, trace	0.95	no
orchestration	workflow completion	state, trace	0.95	no
efficiency	efficiency	trace	0.9	no
quality	answer quality	judge	0.9	no

3 Abstractions

Table 6 fixes the notation; the paragraphs below say what the table cannot.

Outcomes and the baseline. A run the environment broke is not an outcome and is left out. A test with $f_t \geq \frac{1}{2}$ is one the model already fails: a mistake of the model rather than a fault of the system. Such a test cannot reveal anything, so the ground truth belongs to the model: a change the model cannot see, because it ignores the rule with and without the change, is revealed by no test under that model, and a different model may give a different $\mathcal{R}(\Delta)$.

Components and the change set. A component is a part of the system that a change can touch and a run can exercise, named in one vocabulary (Table 7). Comments, docstrings, blank lines, formatting and the order of configuration mappings add nothing: no component, no file and no diff line, so no strategy and no probability can depend on them. Skill files are prompts the model reads word for word, so all of their text counts.

Table 4: What a trace records, and what each record feeds.

record	feeds
the agents started and the skills loaded	agent and skill dependencies; coverage of agents, skills and workflows
the tools called, with their status	tool dependencies; the tool error paths covered; the trace assertions
the configuration keys read	configuration dependencies
the code functions and lines executed, each line attributed to the test and repetition that ran it	code dependencies at line level; the reveal ratios of Section 4.4
the plan, the delegations and the approvals, as span events	the order of agents, the hand-offs of ids and the approvals the workflow tests check

Table 5: The 43 controlled changes.

category	count	examples
code	10	argument swap in a tool adapter; retries drop idempotency keys; approval gate bypassed; external links treated as reachable; a shared read reports the wrong review flag; ranking sorted the wrong way; case-sensitive search and brand check
runtime	3	the agent loop stops six steps early; a delegated answer is truncated; a delegation drops the ids of earlier steps
skill	11	publishing drops link validation or the permission check; authoring drops its metadata step; translation hard-codes a default; rollback restores the wrong version; governance drops the consent check; the routing skill stops asking for approval, or drops the brand gate
agent definition	2	an agent loses a tool
configuration	7	link validation disabled; review not required; a step budget of five; no retries; a title limit of twenty; another default agent; no approval needed to take a page down
combined	1	a keyword rule in code and a very low limit in configuration
benign	9	prose reflowed or reworded; a comment or docstring edited; local variables renamed; configuration comments reordered; an unused tool added; prompt wording; lists reordered

Dependencies. A component matches a dependency set when the set holds the component, its base (`skill:x` for `skill:x#procedure`), or a function of a module changed at module level. A test's behaviour signature is the tuple of agents, tools, fault profile and dataset of its first baseline run; tests with the same signature are near-duplicates.

Strategies. A strategy sees $C(\Delta)$, the baseline facts about every test (D_t^{obs}, ϕ_t, $c(t)$, f_t, ρ_t, $\text{props}(t)$, D_t^{decl} and the test's text), and the outcomes of earlier changes on which it may calibrate, but never $\mathcal{R}(\Delta)$. It chooses a test, observes its outcome and cost on V_1, and either chooses again or stops, within B. A non-adaptive strategy is an order of T run greedily within the budget. An adaptive one may also return a verdict, CLEAN or REGRESSION, and a probability.

Metrics. Table 8. Prioritization alone, with every test run and no stopping, is scored by APFD [25]. With $n = |T|$, $m = |\mathcal{R}(\Delta)|$ and TF_i the position in the order of the test that reveals fault i,

$$\text{APFD} = 1 - \frac{\sum_{i=1}^{m} TF_i}{n\,m} + \frac{1}{2n}, \tag{1}$$

Table 6: Notation.

symbol	meaning
V_0, $V_1 = V_0 + \Delta$	the current version and the changed version
T, $c(t)$	the suite of tests and eval items; the cost of t in tokens, prompt and completion over every model call it makes
$\rho_t \in \{1, 2, 3, 4\}$	the risk weight of t
$\text{props}(t)$, $\mathcal{P}$	the properties t checks; the catalog of properties
D_t^{decl}	the dependencies t's author declared
$X_t(V)$, f_t	the outcome of t on V, pass or fail, random because a model produces it; t's fail rate over K runs on V_0
$C(\Delta)$, $k(c)$	the change set, the components the diff of V_0 and V_1 touches (Table 7); the kind of a component
$\phi_t(d)$, D_t^{obs}	the share of t's traced runs on V_0 that exercised component d; the components with $\phi_t(d) > 0$
$\mathcal{R}(\Delta)$	the tests that reveal Δ: a majority of their K runs pass on V_0 and fail on V_1; Δ is a regression when $\mathcal{R}(\Delta) \neq \emptyset$
$\mathcal{P}(\Delta)$	the affected properties: $C(\Delta)$ touches one of their sources, or a test that checks them exercised a changed component on V_0
B	the budget in tokens, given as a share of $\sum_{t \in T} c(t)$

Table 7: The component vocabulary, shared by change sets and dependencies.

component	what it names
`code:<file>::<function>`	a function whose normalized syntax tree changed
`line:<file>:<n>`	a line of code
`tool:<name>`	a registered tool adapter
`skill:<name>#procedure`, `#prose`, `#meta`	the three parts of a skill file
`agent:<name>.<field>`	a field of an agent definition; when a tool or skill list only lost items, the component records which, so that only the tests that used one of them are reached
`config:<key>`	an effective setting
`fixture:<name>`	the platform's starting data

which is near 1 when the first tests find every fault and near 0 when the last ones do; a random order scores about $\frac{1}{2}$. Because costs differ by an order of magnitude, we also score its cost-cognizant form [6, 16]. With c_j the cost of the j-th test in the order, $C = \sum_j c_j$, and σ_i the severity of fault i (the risk weight of its revealing test),

$$\text{APFD}_c = \frac{\sum_{i=1}^{m} \sigma_i \left(C - \sum_{j<TF_i} c_j - \frac{1}{2} c_{TF_i}\right)}{C \sum_{i=1}^{m} \sigma_i}. \tag{2}$$

Finding a fault after a few cheap tests costs little and scores high; finding it after the long workflows scores low, however few tests came before. An order can score well on APFD and poorly on APFD_c by putting the expensive tests first.

4 The strategies

Every strategy answers the same question, which tests to run on V_1 and in what order, from the inputs listed in Section 3. Table 9 says what each one reads and whether it stops, Table 10 lists every constant, and Table 11 what each is good and bad at. Five strategies are change-aware, in that they read $C(\Delta)$ or the diff; the two reproduced methods do not. Figure 2 follows the proposed method from the diff to a verdict. Brackets $[\cdot]$ are 1 when the condition inside holds and 0 otherwise.

Table 8: The metrics, for one change and one run of a strategy.

metric	definition	needs $\mathcal{R}(\Delta)$
detection	a revealing test was run and failed	yes
cost to first detection	the tokens spent up to that failure	yes
tokens spent	the cost of every test run, on every change	no
false alarm	a failure of a test outside $\mathcal{R}(\Delta)$; on a change no test reveals, a failure the change did not cause	yes
verdict correct	the verdict agrees with $\mathcal{R}(\Delta) \neq \emptyset$	yes
change coverage	the shares of $C(\Delta)$ exercised, of $\mathcal{P}(\Delta)$ checked, and of the affected workflows and features reached; whether every mandatory property in $\mathcal{P}(\Delta)$ got a test	no
APFD, APFD$_c$	how early, by position or by cost, a complete order finds each fault	yes

Table 9: The strategies compared. Every non-adaptive order runs greedily within the budget and never stops.

strategy	reads	does	after
`full`	nothing	the suite in catalog order	reference
`random`	nothing	a random permutation	reference
`declared`	$C(\Delta)$; D_t^{decl}, ρ_t, $c(t)$	selects the tests whose declared dependencies match	[24]
`ir_tfidf`	the diff's text; the tests' text	orders the suite by TF-IDF similarity to the diff	REPiR [26]
`observed`	$C(\Delta)$; D_t^{obs}, ϕ_t, ρ_t, $c(t)$	selects the tests whose traces reached the change	this paper
`cost_aware`	$C(\Delta)$; D_t^{obs}, ϕ_t, f_t, ρ_t, $c(t)$	orders the suite by affinity, risk and cost	[6, 17]
`casper_slices`	baseline behaviour signals, pass rates, $c(t)$	probes representatives of behavioural slices, then runs the slices by drift	CASPER [19]
`risk_aware_ts`	a context vector and an impact per test, $c(t)$	a contextual bandit, one test per round	[15]
`adaptive_bayes`	$C(\Delta)$ with kinds; every baseline fact; earlier changes with their outcomes	the most information per token; stops with a verdict	this paper

4.1 Four non-adaptive orders

Algorithm 1 gives the four change-aware orders that need no model of the outcomes. Two select a subset and order it, `declared` on what the authors wrote down and `observed` on what the traces show; two order the whole suite, `ir_tfidf` by text alone and `cost_aware` by affinity, risk, history and cost.

Table 10: The constants of the strategies.

strategy	constant	value	role
`adaptive_bayes`	π_c	0.45–0.5 for code, skill procedures, configuration and agent fields; 0.12–0.15 for prompt wording and skill prose	prior that a changed component is broken, by kind
	r_k^0	0.12 changed line; 0.03 function only called; 0.30 skill procedure; 0.30 agent field; 0.12 configuration key; 0.05 prose	default reveal ratio per kind, toward which the calibrated ratio is padded
	m_0	2	padding of the reveal ratio and of the lift
	s_{ind}; clip	0.02; $[0.02, 0.9]$	sensitivity of a test with no link to the component; bounds of Equation 4
	γ	0.25	risk tilt $1 + \gamma(\rho_t - 1)$ of the information gain
	δ	0.35	discount of the near-duplicates of a test that passed
	ε	0.05	stop with CLEAN when $1 - P(\mathbf{0}) < \varepsilon$
	$\lvert C(\Delta)\rvert$	at most 8	the likeliest components are kept when there are more
`casper_slices`	n_s, $n_{\min}$, w, K_r	8, 4, 3, 2	slice size, minimum size, outcome weight, representatives per slice
`risk_aware_ts`	α, λ	2, 1	weight of the regression's estimate in the Beta prior; ridge penalty

Algorithm 1 The four non-adaptive change-aware orders

procedure DECLARED($C(\Delta)$; D_t^{decl}, ρ_t, $c(t)$ for every t)
  $S \leftarrow \{t : D_t^{\text{decl}}$ matches some $c \in C(\Delta)\}$; $a_t \leftarrow$ the number of components matched
  return S sorted by ρ_t descending, then a_t descending, then $c(t)$ ascending
end procedure
procedure IR_TFIDF(the text of the change; the text of every t)
  $q \leftarrow$ TF-IDF vector of the change text over the vocabulary of the tests' texts; $v_t \leftarrow$ TF-IDF vector of the text of t
  return T sorted by $\cos(q, v_t)$ descending
end procedure
procedure OBSERVED($C(\Delta)$; D_t^{obs}, ϕ_t, ρ_t, $c(t)$ for every t)
  $S \leftarrow \{t : D_t^{\text{obs}}$ matches some $c \in C(\Delta)\}$; $a_t \leftarrow \sum_{c \text{ matched}} \phi_t(c)$, line hits counted once per file
  return S sorted by $a_t\, \rho_t / c(t)$ descending
end procedure
procedure COST_AWARE($C(\Delta)$; D_t^{obs}, ϕ_t, f_t, ρ_t, $c(t)$ for every t)
  $a_t \leftarrow$ as in `observed`, 0 for an unaffected test; $e_t \leftarrow [t$ executed a changed line$]$
  return T sorted by $\rho_t\,(0.05 + a_t + e_t)\,(1 + f_t)/c(t)$ descending
end procedure

Table 11: What each strategy is good and bad at.

strategy	good at	bad at
`declared`	cheap, explainable, safe when the metadata is complete	caps at what the authors declared, which leaves out what Section 2 says nobody declares, so it tops out whatever the budget
`ir_tfidf`	no instrumentation; strong wherever tests, skills and settings share vocabulary	never stops, so it spends the whole budget on every change; blind to code reached without a shared word
`observed`	selects everything the traces show the change can reach, from one traced run per test	does not stop within its selection; treats a covered component as an affected one, so it spends on tests that execute the changed line without being able to notice the change
`cost_aware`	affected tests first, cheap before expensive, the rest after them, so the budget is always used	never stops; its history factor $(1 + f_t)$ pushes forward the tests the model already fails, which it then reports as failures the change did not cause
`casper_slices`	needs no diff, so it applies to a model upgrade, a provider's silent update or a prompt-level change in a system whose code the testing team does not own; a fixed number of representatives per slice samples a large suite as cheaply as a small one; the slices are a map of the suite's behaviours	blind to an edit, so an edit gives it no signal until the representatives have run; no stopping rule; a representative the model answers unstably drags its whole slice forward
`risk_aware_ts`	learns from the outcomes it sees and weighs them by impact, so it finds the high-risk failures first once a few have been seen	any failure is a reward, so a test the model already fails counts as a find; ignores the diff; with no outcome yet it has only the risk to go on; no stopping rule, no verdict
`adaptive_bayes`	selection, prioritization and stopping from one model: returns the budget it does not need, gives a verdict with a probability, discounts the tests the model already fails, tests every affected mandatory property before calling a change clean	a narrow regression, which few tests reveal, where it can become confident that nothing broke before reaching them (Section 5 counts these); needs a traced baseline and a record of earlier changes; its stopping rule assumes greedy decoding

4.2 CASPER's slices, reproduced: `casper_slices`

CASPER [19] mines signals from the logs of the current version, cuts the suite into slices that are coherent in signal space and consistent in outcome, runs a few representatives of every slice on the new version, and ranks the slices by how much their behaviour moved. Here (Algorithm 2) the signals are ten per test (turns, tool calls, distinct tools, the share of the most-called tool, the entropy of the calls, hand-offs, skills loaded, settings read, the size of the run and the share the model wrote); the slices come from k-means [12] with the baseline outcome as a heavier column, in place of the paper's evolutionary search; and, because a replay (Section 5) returns an outcome and a cost, not a trace, a slice is scored by the shift of its representatives' failure rate. For the prioritization metrics its order is that of a run without a budget limit in which every test passes.

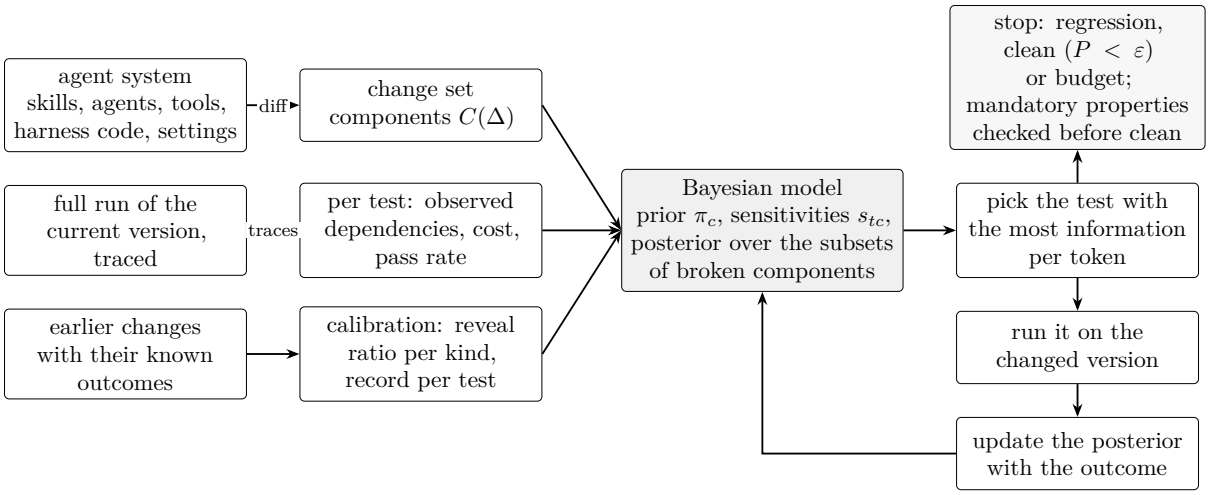


Figure 2: From a diff to a verdict. The three inputs on the left exist before the change is tested: the diff, a traced run of the current version, and earlier changes with their known outcomes. The loop on the right runs one test at a time on the changed version.

Algorithm 2 `casper_slices`

Input: the baseline runs of every test, as signals and a pass rate; costs $c(t)$; slice size n_s, minimum size $n_{\min}$, outcome weight w, representatives per slice K_r; budget B

Output: the tests run on V_1, in order

Slice the suite on the current version

$Z \leftarrow$ the standardized signals of every test, with $w\cdot$ (standardized pass rate) as an extra column

slices $\leftarrow$ k-means$(Z, \lceil |T|/n_s \rceil)$; merge every slice smaller than $n_{\min}$ into the slice whose centre is nearest

$\text{reps}_j \leftarrow$ the member of slice j nearest its centre, then farthest-point sampling, K_r in all

Probe the changed version

run reps_j of every slice j on V_1, cheapest slices first, while the budget allows

$\text{shift}_j \leftarrow$ failure rate of reps_j on V_1 minus their failure rate on V_0

Rank the slices by drift, then run them

order the slices: $\text{shift}_j > 0$ by size descending, then $\text{shift}_j < 0$, then unchanged

run the members of each slice in that order, nearest the centre first, until the budget is spent

4.3 Risk-aware adaptive evaluation, reproduced: `risk_aware_ts`

Risk-aware adaptive evaluation [15] is a contextual Thompson-sampling bandit [3, 30]: the reward of a trial is impact times failure; each round, a logistic regression on every outcome so far gives every scenario a failure probability, a Beta posterior per scenario mixes it with the scenario's own outcomes, and one draw per scenario, times the impact, picks the next. Here (Algorithm 3) the scenarios are the tests, each with a context vector x_t (features of its definition and of the shape of its baseline runs, standardized, with an intercept) and an impact $\iota_t = \rho_t$; the budget is in tokens, each test runs once, and σ is the logistic function. For the prioritization metrics its order is static: by impact, cheapest first.

Algorithm 3 `risk_aware_ts`

Input: context x_t, impact ι_t and cost $c(t)$ of every test; mixing weight α, ridge penalty λ; budget B

Output: the tests run on V_1, in order

$n_t \leftarrow 0$, $k_t \leftarrow 0$ for every t; $b \leftarrow B$ ▷ runs and failures seen per test

run one test drawn uniformly among those with $c(t) \le b$; record it as below

while some unrun t has $c(t) \le b$ **do**

fit, sample, pick

$\beta \leftarrow \arg\min_\beta \sum_{t:\, n_t>0} \mathrm{logloss}(\sigma(\beta^\top x_t), k_t) + \frac{\lambda}{2}\|\beta\|^2$ ▷ Newton's method, warm start

$\mu_t \leftarrow \sigma(\beta^\top x_t)$ and $\theta_t \sim \mathrm{Beta}\big(1+\alpha\mu_t + k_t,\ 1+\alpha(1-\mu_t) + n_t - k_t\big)$ for every unrun t with $c(t) \le b$

$t^\star \leftarrow \arg\max_t \iota_t\, \theta_t$; run it on V_1 at cost κ

$n_{t^\star} \leftarrow 1$; $k_{t^\star} \leftarrow$ [failed]; $b \leftarrow b - \kappa$

end while

4.4 The proposed method: `adaptive_bayes`

Hypotheses. Each changed component $c \in C(\Delta)$ is broken with a prior π_c that depends on its kind (Table 10): high for code, skill procedures, configuration and agent fields, low for prompt wording and skill prose. Tool and line components are views of a code change, not separate causes, and are folded into it. The posterior is kept exactly over all $2^{|C(\Delta)|}$ subsets $h \in \{0,1\}^{C(\Delta)}$.

Likelihood. A test passes only if it would pass on the current version and none of the broken components makes it fail:

$$q_t(h) = P(t \text{ fails} \mid h) = 1 - (1 - f_t) \prod_{c:\, h_c=1} (1 - s_{tc}). \tag{3}$$

A test the model already fails (f_t near 1) fails under every hypothesis, so its outcome carries no information, which is what keeps the method from mistaking the model's mistakes for regressions.

Sensitivity. Covering a component is weak evidence by itself: most tests that execute a changed function do not fail when it breaks. The sensitivity is therefore a product of four factors, calibrated (Algorithm 5) from the earlier changes whose outcomes are known, never from the change under test:

$$s_{tc} = \mathrm{clip}\big(r_{k(c)} \cdot \ell_{t,k(c)} \cdot \phi_t(c) \cdot \tau_t,\ 0.02,\ 0.9\big). \tag{4}$$

- r_k is the reveal ratio of the component's kind k: among the tests that statically looked impacted by earlier changes of that kind, the share that revealed them, padded toward a default r_k^0. The kinds separate a test that executed a changed line from one that only called the changed function, and agent fields are calibrated per field.
- $\ell_{tk} = (O_{tk} + m_0)/(E_{tk} + m_0)$ is the test's own lift on that kind: O_{tk} the earlier changes of kind k it revealed, E_{tk} the number an average impacted test would have revealed (the sum of those changes' reveal rates), m_0 a padding that keeps a short record from moving the lift far. A change that breaks most of the suite says little about each test it breaks; a change that one test alone caught says a lot about that test.
- $\phi_t(c)$ is how often the baseline runs exercised the component: for code, the changed line they executed most often, or 1 when they only called the changed function; for an agent that lost tools or skills, how often t used one of the lost items.
- $\tau_t = 0.5 + \mathrm{text}(t, \Delta) + 0.5\,[\text{declared}] \in [0.5, 2]$ tilts the sensitivity up for a test whose text shares vocabulary with the diff and for one whose author declared the dependency; $\mathrm{text}(t, \Delta) \in [0, 1]$ is the cosine of the word sets of the test and of the change, scaled so that the closest test gets 1.

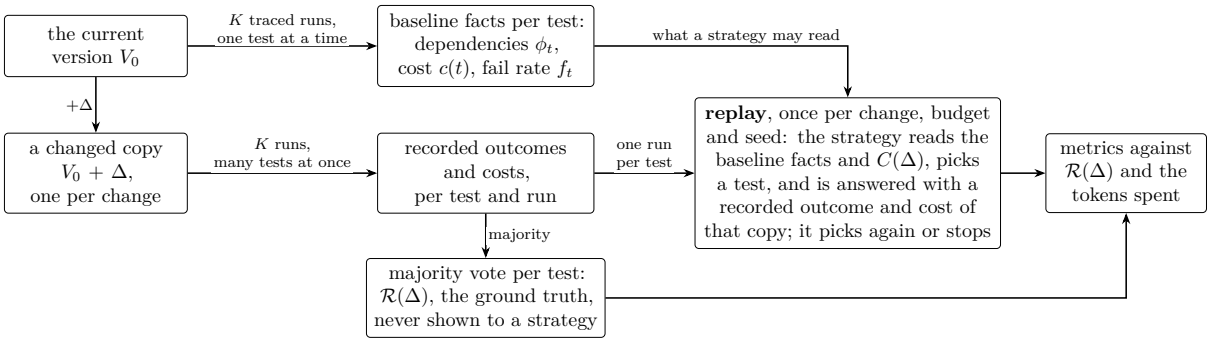


Figure 3: Recording and replay. The top two rows are recorded once per model; the replay below runs every strategy, budget and seed against the recordings.

A test with no link to the component gets the small constant s_{ind}; one declared but never observed gets half the kind's ratio.

Selection and stopping. Algorithm 4 runs the loop:

- *next test*: the one with the largest expected reduction of the posterior's entropy [11] per token, the gain multiplied by $1 + \gamma(\rho_t - 1)$ so that risk tilts the choice. Each test runs at most once: the models decode greedily, so a rerun would almost always repeat the answer;
- *after a pass*: tests with the same behaviour signature as the one that passed have their sensitivities discounted by δ, because near-duplicates give correlated evidence rather than independent draws;
- REGRESSION: at the first failure of a test that passed on the current version, because under greedy decoding that failure is a changed behaviour, which is what the ground truth calls revealing;
- CLEAN: when $1 - P(h = \mathbf{0}) < \varepsilon$; BUDGET: when the budget is spent or no test can still move the posterior;
- *the mandatory rule*: before a run ends without a regression, every mandatory property in $\mathcal{P}(\Delta)$ with no test yet gets the cheapest test of that property that passes on the current version, while the budget allows. A failure there is a regression like any other.

For the prioritization metrics, which score a complete order, the method's order is the greedy sequence it would pick if every test passed, with no stopping, followed by the untouched tests in cost-aware order.

5 Evaluation

Questions.

- **RQ1** Can the tests relevant to a change be identified, and what do declared and observed dependencies each give?
- **RQ2** Can the tests be ordered so that regressions are found earlier?
- **RQ3** How much budget can be returned while detection and coverage hold, and does the stopping rule give the right verdict?
- **RQ4** What do the two published methods, which do not read the change, find on the same testbed?

Figure 3 and Table 12 give the protocol.

Algorithm 4 `adaptive_bayes`: adaptive Bayesian selection with stopping

Input: components $C(\Delta)$ with priors π_c; for every test $c(t)$, f_t, ρ_t, props(t) and s_{tc} (Algorithm 5); near-duplicate groups; the affected mandatory properties; budget B; ε, δ, γ

Output: the tests run R, a verdict, and $1 - P(\mathbf{0})$, the probability that some component is broken

$P(h) \propto \prod_c \pi_c^{h_c}(1-\pi_c)^{1-h_c}$ for $h \in \{0,1\}^{C(\Delta)}$ ▷ $h = \mathbf{0}$: nothing is broken
$R \leftarrow \emptyset$; $b \leftarrow B$
loop
  if $1 - P(\mathbf{0}) < \varepsilon$ **then return** FINISH(CLEAN)
  end if
  $t^\star \leftarrow$ PICK; **return** FINISH(BUDGET) if there is none
  if OBSERVE$(t^\star)$ = REGRESSION **then return** REGRESSION
  end if
end loop

function PICK
  $p_t \leftarrow \sum_h P(h)\, q_t(h)$ for every unrun t ▷ $P(t$ fails$)$; q_t from Equation 3
  $g_t \leftarrow H(P) - p_t\, H(P \mid t \text{ fails}) - (1-p_t)\, H(P \mid t \text{ passes})$ ▷ expected information gain
  return $\arg\max\{\, g_t\,(1+\gamma(\rho_t-1))/c(t) : t \notin R,\ c(t) \le b,\ g_t > 0 \,\}$
end function

function OBSERVE(t)
  run t on V_1: outcome x, cost κ; $R \leftarrow R \cup \{t\}$; $b \leftarrow b - \kappa$
  $P(h) \leftarrow P(h)\, q_t(h)$ if $x =$ fail, else $P(h)\,(1-q_t(h))$; normalize
  if $x =$ fail and $f_t < \frac{1}{2}$ **then return** REGRESSION ▷ a test that used to pass has failed
  end if
  if $x =$ pass **then** $s_{uc} \leftarrow \delta\, s_{uc}$ for every unrun u in the group of t ▷ correlated evidence
  end if
  return x
end function

function FINISH(verdict)
  for all affected mandatory properties p that no test in R checks, in id order **do**
    $u \leftarrow$ the cheapest $u \notin R$ with $p \in$ props(u), $f_u < \frac{1}{2}$, $c(u) \le b$; skip p if there is none
    if OBSERVE(u) = REGRESSION **then return** REGRESSION
    end if
  end for
  return verdict, with $1 - P(\mathbf{0})$
end function

Table 12: The protocol.

repetitions K	5 per version
recording	the current version one test at a time, so that coverage can follow each test; each of the 43 changed copies many tests at once
budgets	2, 5, 10, 20, 30, 50 and 100% of the full-suite tokens
seeds	5; a seed chooses which recorded run answers each test and drives the strategies that draw at random
calibration	on the other 42 changes, never on the one under test
replays	1,505 per strategy: 43 changes, 7 budgets, 5 seeds

Algorithm 5 Calibration of the sensitivities from the earlier changes

Input: earlier changes $\mathcal{D}$ with their revealing sets $\mathcal{R}(\Delta')$; defaults r_k^0 per kind; padding m_0; the change under test $\Delta \notin \mathcal{D}$; for every test ϕ_t, D_t^{decl} and its text

Output: s_{tc} for every test t and component $c \in C(\Delta)$

Learn from the record

for all $\Delta' \in \mathcal{D}$ and components $c' \in C(\Delta')$ of kind k **do**

  $I \leftarrow$ the tests that look impacted by c': executed a changed line, called the changed function, or match the component

  rate $\leftarrow |\mathcal{R}(\Delta') \cap I|/|I|$; append rate to A_k

  O_{tk} += $[t \in \mathcal{R}(\Delta')]$ and E_{tk} += rate for $t \in I$ ▷ once per change and kind

end for

$r_k \leftarrow \text{clip}((\sum A_k + m_0\, r_k^0)/(|A_k| + m_0),\ 0.01,\ 0.9)$ for every kind k

Apply to the change under test

for all tests t and components $c \in C(\Delta)$ of kind k **do**

  $\ell_{tk} \leftarrow (O_{tk} + m_0)/(E_{tk} + m_0)$; $\tau_t \leftarrow 0.5 + \text{text}(t, \Delta) + 0.5\,[\text{declared}]$

  $s_{tc} \leftarrow \text{clip}(r_k\, \ell_{tk}\, \phi_t(c)\, \tau_t,\ 0.02,\ 0.9)$ if $\phi_t(c) > 0$; $\frac{1}{2} r_k$ if declared only; s_{ind} otherwise

end for

Ground truth. On the current version 172 of 180 tests pass. The eight failures are the model's own mistakes, which clearer wording of the requests does not fix (Table 13). Of the 34 regressions, 29 are revealed by at least one test: the five others remove a guard the model does not act on with or without the change, so it behaves the same, wrongly, on both versions (Table 14). No benign change fails a test. Across the 43 changes the five runs disagreed on four (change, test) pairs, which the majority vote absorbs. The 29 revealed regressions and the 14 changes no test reveals are the population below.

Table 13: The eight tests the model fails on the current version.

test	what the model does
PUB-10, WF1-01	requests the review, as it should, then schedules the page anyway
AUT-04	after an edit, requests a review nobody asked for
CFG-01	requests a review on a platform that does not require one
SEO-02	applies an 83-character title over the 60-character limit
WF2-03	sends three pages to publishing in one step; publishing stops at the broken one, and it reports all three as republished
WF5-02	after the restored version fails its link check, sends the page to publishing anyway; publishing's own check refuses it
WF6-02	after the export is refused for lack of consent, stops the whole plan, although the other steps do not depend on it

Table 14: The five regressions no test reveals under Gemma.

change	guard removed	why no test reveals it
review not required (configuration); an edit keeps the review flag (code)	the review flag	it schedules without an approved review and requests reviews nobody asked for, with and without the change
a title limit of twenty (configuration); the SEO skill without its length guard	the title length limit	it applies the over-long title whatever the limit
link validation that ignores asset references (skill)	a non-blocking asset reference	it stops at a broken asset reference even when the skill says it does not block

Table 15: Detection of the 29 revealed regressions by nominal budget (share of the full-suite tokens), and the tokens actually spent: on regressions at a 20% and a 100% budget, and on the 14 changes no test reveals at 100%. Five seeds; the last three columns are shares of the full-suite tokens.

strategy	2%	5%	10%	20%	50%	100%	spent@20	spent@100	unrevealed@100
`full`	0.00	0.14	0.24	0.34	0.90	1.00	0.20	0.98	1.00
`random`	0.09	0.22	0.37	0.56	0.86	1.00	0.20	0.98	1.00
`declared`	0.45	0.59	0.55	0.62	0.62	0.62	0.07	0.11	0.14
`ir_tfidf`	0.59	0.69	0.76	0.93	1.00	1.00	0.20	0.98	0.99
`observed`	0.48	0.62	0.69	0.86	0.86	1.00	0.15	0.49	0.33
`cost_aware`	0.38	0.55	0.62	0.86	0.86	1.00	0.20	0.98	0.99
`casper_slices`	0.17	0.28	0.52	0.59	0.90	1.00	0.20	0.98	1.00
`risk_aware_ts`	0.11	0.28	0.35	0.57	0.83	1.00	0.20	0.98	1.00
`adaptive_bayes`	0.61	0.74	0.78	0.78	0.81	0.81	0.03	0.03	0.20

Table 16: Order quality and verdicts. APFD and APFD_c of the complete order; cost to the first true detection as a share of the full suite (detected cases); detection of the 21 regressions that break a safety property at a 10% and a 20% budget; share of replays, over all budgets, in which the strategy saw a failure on one of the 14 changes no test reveals.

strategy	APFD	APFD_c	first detection	safety@10	safety@20	false alarm
`full`	0.63	0.58	0.19	0.14	0.24	0.72
`random`	0.54	0.54	0.14	0.41	0.60	0.69
`declared`	0.81	0.76	0.028	0.52	0.57	0.58
`ir_tfidf`	0.78	0.77	0.040	0.86	1.00	0.71
`observed`	0.75	0.79	0.049	0.57	0.81	0.58
`cost_aware`	0.74	0.77	0.060	0.57	0.81	0.94
`casper_slices`	0.58	0.52	0.12	0.62	0.71	0.57
`risk_aware_ts`	0.60	0.60	0.14	0.47	0.71	0.76
`adaptive_bayes`	0.83	0.81	0.017	0.84	0.84	0.00

Selection (RQ1). Declared metadata caps detection at 62% however large the budget (Table 15), because shared skills reached through an include, harness code and most configuration keys are not declared. The observed dependencies select every revealing test for half the suite's tokens and reach 86% within a 20% budget. The argument for trace-informed selection holds.

Prioritization (RQ2). The change-aware strategies raise APFD_c from 0.54 (random) to 0.76–0.81 (Table 16). The adaptive method's order is the best and finds the first regression soonest, after 1.7% of the suite's tokens. The two reproduced methods order near random (0.52 and 0.60 against 0.54) and find their first regression after 12–14% of the suite, where the change-aware strategies need under 2% to 6%.

Budget and verdicts (RQ3). The adaptive method is the only strategy that stops; `declared` and `observed` return budget only by running out of selected tests. It finds 74% of the regressions at a 5% budget and 78% at 20%, for under 3% of the tokens, and spends a fifth of the suite on a change no test reveals even with the whole budget (Table 15). TF-IDF over the diff is the strongest selector at 20% (93%) and the strongest on the safety-breaking changes (all of them), and it spends the whole budget every time, on changes no test reveals too. The adaptive method's ceiling is 81%: 145 of the 1,015 replays of regressions ended CLEAN, all on regressions that one to five tests reveal, where it became confident that nothing broke before reaching them. Table 17 lists what it misses at 20%.

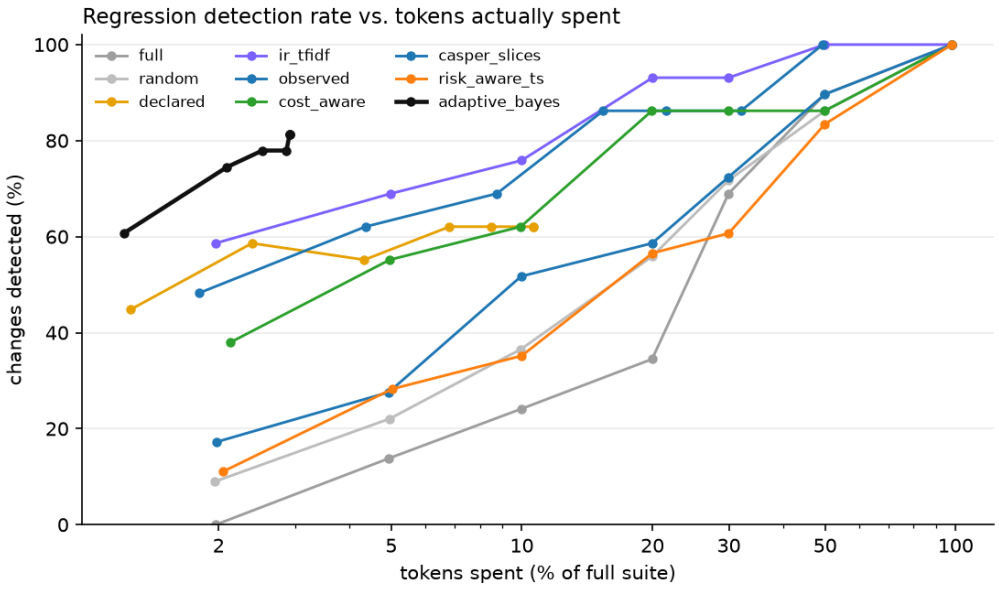


Figure 4: Detection of the 29 revealed regressions against the tokens actually spent, every strategy and budget. The adaptive method sits at the left: it stops.

Table 17: The regressions the adaptive method misses at a 20% budget: how many tests reveal each, in how many of the five seeds a revealing test failed, and how the run ended.

change	revealing tests	caught	how the run ended
a delegation drops the ids of earlier steps	4	0 of 5	CLEAN: confident before reaching them
retries drop idempotency keys	5	0 of 5	CLEAN
the agent loop stops six steps early	5	0 of 5	CLEAN
the authoring skill drops its metadata step	1	0 of 5	CLEAN
the routing skill drops the brand gate	1	0 of 5	CLEAN
the combined change	1	0 of 5	no test could still move the estimate
the governance skill drops the consent check	5	3 of 5	REGRESSION in every seed, on a revealing test in three

The eight tests that fail on the current version fail on nearly every changed copy too. A strategy that runs one sees a failure the change did not cause: on the 14 changes no test reveals, this happens to `full` in 72% of its replays, to `cost_aware` in 94%, to the bandit in 76% and to CASPER's slices in 57% (Table 16). The adaptive method discounts a test by its failures on the current version (Equation 3) and stops only on a test that used to pass, so in 490 replays on these changes it neither ran a test that failed (Table 16) nor gave a REGRESSION verdict.

Live against replay. The prototype, which runs the adaptive method with the model live instead of looking outcomes up, was run on all 43 changes at a 20% budget, one check at a time. On 42 it gave the verdict of the replay's five seeds, with the same tests and the same tokens to within a few hundred. The exception is a live false alarm on the benign rewording of the orchestrator's prompt: a workflow test failed live, with the orchestrator skipping an approval, where its five recorded runs pass. That is the one-off difference that greedy decoding under batching leaves, on a test whose outcome hangs on one decision of the model. The 43 checks took 21 minutes of model time: six tests and 1.9% of the suite's tokens per check at the median.

The replay is a faithful stand-in for the live prototype, and the one false alarm is the noise floor the stopping rule would have to tolerate with repetitions.

The text factor. The tilt τ_t can use word overlap (the default), the TF-IDF similarity of `ir_tfidf`, or no text at all (Table 18). TF-IDF brings forward the one test that reveals a narrow change and gains up to five points while spending less; without any text the method loses ten to seventeen points. Text matters, and coverage and history carry most of the result. Word overlap stays the default until a second application, with tests and changes worded differently, shows whether TF-IDF's gain holds there too.

Table 18: Detection by the method with three text measures, with the tokens spent in parentheses, against `ir_tfidf` alone.

budget	word overlap	TF-IDF	no text	`ir_tfidf` alone
5%	0.74 (2.1%)	0.79 (1.9%)	0.57 (2.4%)	0.69 (5%)
20%	0.78 (2.9%)	0.83 (2.4%)	0.68 (3.5%)	0.93 (20%)
50%	0.81 (2.9%)	0.83 (2.4%)	0.68 (4.6%)	1.00 (50%)

The mandatory rule. Table 19. The rule is also a detector: it catches the publishing skill without its link validation, which the method without the rule calls clean, because the method must test the mandatory property "no broken publish" before ending, and that test fails. The rule never caused a false verdict. Below a 10% budget it often cannot afford the extra test.

Table 19: The mandatory rule at a 20% budget.

	without the rule	with the rule
runs that ended without a regression having tested every affected mandatory property	33%	90%
extra tests per change	0	0.8
detection	74%	78%

The reproduced methods (RQ4). CASPER's slices and the risk-aware bandit find 59% and 57% of the regressions at a 20% budget, close to random (56%) and far from the strategies that read the diff and the traces (78–93%; declared metadata alone 62%). The bandit's impact is the test's risk, so it does better on the changes that break a safety property (71% at 20% and 98% at 50%, against 57% and 83% over all regressions), and it rewards any failure, so it runs the tests the model already fails early (76% false alarms, Table 16). Neither stops. These numbers measure what the diff is worth, not CASPER in its own setting (Table 11): a platform sees changes with a diff, which the change-aware strategies cover, and model or provider updates without one, where CASPER's representatives are the natural first pass. The two could be combined: the slices can serve as the near-duplicate groups of the adaptive method, and a slice whose representatives drifted can enter the model as a changed component. On this testbed every change has a diff and the suite is small (180 tests, twelve slices), which is the case slicing has least to offer.

Other models. The same application was recorded with Qwen3.8-27B and with gpt-oss-120b, served as they are, with greedy decoding and five repetitions, like Gemma (Table 20). Neither recording is a clean ground truth, and the five repetitions do not settle either vote: every benign change is "revealed" by a few tests, the change whose change set is empty included. Seven tests carry Qwen's variation: one of the two cases a model judges against a rubric, the title at the length limit, four long workflows and one authoring case. Thirteen tests carry gpt-oss's; two of

them pass on the current version and fail on nearly every changed copy, whichever the change. The baseline runs one test at a time and the changed copies many at once. Recording the empty change one test at a time, five repetitions, separates batching from the model: under Qwen the runs disagree on one test, which sits at the majority line in the baseline too, and flip no other test against the baseline, where the same change recorded 64 tests at a time has nine tests whose runs disagree and six that flip. Qwen's variation is batching; one test at a time it is as repeatable as Gemma, whose vote is stable either way (four disagreeing pairs, above). vLLM's batch-invariant mode, which would make batched decoding exactly repeatable, does not support Qwen's attention layers, and the empty change recorded in batches, taken as the reference, still leaves every other benign change flipping tests (two to five under Qwen, one to four under gpt-oss). gpt-oss was not recorded one test at a time.

Table 20: The three recordings. Tests that pass on the current version; tests that "reveal" a benign change, under the majority over five runs and over the first three; (change, test) pairs on which the two votes differ; pairs on which a change's own five runs disagree; tests that carry the variation.

model	pass on V_0	benign "revealed", 5 / 3 runs	votes differ	runs disagree	varying tests
Gemma 4 E4B	172	0 / 0	2	4	0
Qwen3.8-27B	173	3–6 / 2–4	75	357	7
gpt-oss-120b	153	2–5 / 1–4	95	577	13

Threats to validity.

- The regressions are mutants, not real changes; they were written to resemble regressions a team would ship, and the testbed does not yet pair them with a version history. The tests and the changes were written by the same people.
- One primary application, built for the purpose, and one model with a stable ground truth; the second and third models show that the ground truth depends on the model and on how it is served.
- The testbed is textual, which favours the text factor and `ir_tfidf`; a second application with differently worded tests and changes (tau-bench [1, 32]) is future work (Section 7).
- The calibration uses the other recorded changes, so leakage is controlled by leaving the change under test out, but the 42 training changes are few, which is why every learned ratio is padded toward a default.
- The stopping rule assumes greedy decoding; with sampling, a single failure would need repetitions [2, 18].

6 Take-aways

What the experiments say, in the order a team would act on them.

1. **Record traces; do not maintain a map.** Hand-written dependency metadata misses what shared skills, harness code and settings reach; dependencies read from traces recover it (RQ1). One instrumentation of the agent runtime replaces a test-to-component map that nobody keeps current.
2. **Read the diff.** The strategies that read the diff and the traces find most regressions early; the two that do not, CASPER's slices and the risk-aware bandit, sit near a random order and find their first revealing test late (RQ2, RQ4).
3. **Stop when confident, with a safety net.** The adaptive method finds more than any other strategy at equal tokens spent, and most of what the best full-budget strategy finds, for a small fraction of its tokens; it returns the rest. Its price is a ceiling on narrow regressions (RQ3). The mandatory rule covers the safety properties for under one extra test

per change and catches a change the stopping rule alone misses. The catalog of properties is where that decision lives: whoever sets the mandatory flag decides what the pipeline never skips.

4. **Discount the tests the model already fails.** Every other strategy reports some of the model's own failures as the change's (RQ3). A pipeline should keep each test's fail rate and act only on a flip.
5. **Measure the model's noise before trusting a verdict.** An empty change, recorded like the others and once more one test at a time, measures the noise floor of a model under a serving setup and says whether it is the batch or the model (Table 20); record it before trusting a majority vote or a stopping rule.
6. **Text is a strong, cheap baseline.** TF-IDF similarity between the diff and the tests' text has the best detection when the whole budget is spent, and it never stops; it is the fallback order to keep in any pipeline.

7 Conclusion and future work

After a change to an agent system, the diff is worth reading. Mapped to tests through the dependencies observed in traces, it finds most of the regressions within a fifth of the suite's tokens, where hand-written metadata and the two closest published methods, which do not read it, find far fewer. A Bayesian model over the changed components, with sensitivities calibrated from the record of earlier changes, turns selection, prioritization and stopping into one decision: it returns most of the budget, orders the suite best, gives no false verdict on the changes no test reveals, and tests every mandatory property a change may affect before it calls the change clean. The price is a ceiling on narrow regressions, which the mandatory rule lowers for the properties that matter most.

Three things come next:

1. a second system under test, tau-bench's airline domain [1, 32], with the same open model playing both the agent and the user, to see whether the text factor and the calibration hold where tests and changes are worded differently;
2. real version histories beside the mutants, with the changes a team shipped and the tests that caught them, so that the reveal ratios are learned from practice;
3.
4. a ground truth for Qwen with the changed copies recorded one test at a time, and the repetitions that a model whose answers vary under identical conditions needs, with the empty change as the instrument: how many runs, under which serving setup, before a majority vote is a ground truth and a single failure is a verdict.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: evaluating conversational agents in a dual-control environment, 2025. arXiv:2506.07982.

[2] Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. On randomness in agentic evals, 2026. arXiv:2602.07150.

[3] Olivier Chapelle and Lihong Li. An empirical evaluation of Thompson sampling. In *Advances in Neural Information Processing Systems 24 (NeurIPS)*, 2011.

[4] Eunho Cho, Donghwan Shin, and In-Young Ko. Evolution-aware regression test prioritization of ML-enabled systems using gradient-based behavior vectors, 2026. arXiv:2606.28037.

[5] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: help for the practicing programmer. *IEEE Computer*, 11(4):34–41, 1978.

[6] Sebastian Elbaum, Alexey G. Malishevsky, and Gregg Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 329–338, 2001. doi: 10.1109/ICSE.2001.919106.

[7] Sebastian Elbaum, Alexey G. Malishevsky, and Gregg Rothermel. Test case prioritization: a family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2):159–182, 2002. doi: 10.1109/32.988497.

[8] Sebastian Elbaum, Gregg Rothermel, and John Penix. Techniques for improving regression testing in continuous integration development environments. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 235–245, 2014. doi: 10.1145/2635868.2635910.

[9] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[10] Peiyu Li, Xiuxiu Tang, Si Chen, Ying Cheng, Ronald Metoyer, Ting Hua, and Nitesh V. Chawla. Adaptive testing for LLM evaluation: a psychometric alternative to static benchmarks, 2025. arXiv:2511.04689.

[11] Dennis V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956.

[12] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[13] Wanqin Ma, Chenyang Yang, and Christian Kästner. (Why) is my prompt getting worse? Rethinking regression testing for evolving LLM APIs. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering: Software Engineering for AI (CAIN)*, 2024. arXiv:2311.11123.

[14] Mateusz Machalica, Alex Samylkin, Meredith Porth, and Satish Chandra. Predictive test selection. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 91–100, 2019. doi: 10.1109/ICSE-SEIP.2019.00018.

[15] Priyanath Maji and Spandan Ghose Chowdhury. Risk-aware adaptive evaluation: finding high-impact failures under limited budgets, 2026. URL `https://arxiv.org/abs/2609.38914`. arXiv:2609.38914; NeurIPS 2026 workshop on the evaluation of interactive agents.

[16] Alexey G. Malishevsky, Joseph R. Ruthruff, Gregg Rothermel, and Sebastian Elbaum. Cost-cognizant test case prioritization. Technical Report TR-UNL-CSE-2006-0004, University of Nebraska–Lincoln, 2006.

[17] Atif Memon, Zebao Gao, Bao Nguyen, Sanjeev Dhanda, Eric Nickell, Rob Siemborski, and John Micco. Taming Google-scale continuous testing. In *Proceedings of the 39th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 233–242, 2017. doi: 10.1109/ICSE-SEIP.2017.16.

[18] Evan Miller. Adding error bars to evals: a statistical approach to language model evaluations, 2024. arXiv:2411.00640.

[19] Biruk Asmare Muse, Lionel Briand, Yiwei Lu, and Keheliya Gallaba. CASPER: change-aware slice prioritization for efficient regression testing of LLM-based systems, 2026. URL `https://arxiv.org/abs/2608.00378`. arXiv:2608.00378.

[20] Franck Ndzomga. Efficient benchmarking of AI agents, 2026. arXiv:2603.23749.

[21] OpenTelemetry. Semantic conventions for generative AI: agent spans. `https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-agent-spans.md`, 2026.

[22] Rongqi Pan, Taher A. Ghaleb, and Lionel C. Briand. LTM: scalable and black-box similarity-based test suite minimization based on language models. *IEEE Transactions on Software Engineering*, 50(11), 2024. arXiv:2304.01397.

[23] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.

[24] Gregg Rothermel and Mary Jean Harrold. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology*, 6(2):173–210, 1997. doi: 10.1145/248233.248262.

[25] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. doi: 10.1109/32.962562.

[26] Ripon K. Saha, Lingming Zhang, Sarfraz Khurshid, and Dewayne E. Perry. An information retrieval approach for regression test prioritization based on program changes. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 268–279, 2015. doi: 10.1109/ICSE.2015.47.

[27] Zahra Seyedghorban, Egor Klimov, Arie van Deursen, Annibale Panichella, and Burcu Kulahcioglu Ozkan. Observability and fault injection for LLM-based multi-agent systems in software engineering, 2026. arXiv:2608.24271.

[28] Helge Spieker, Arnaud Gotlieb, Dusica Marijan, and Morten Mossige. Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 12–22, 2017. doi: 10.1145/3092703.3092709.

[29] Gou Tan, Zhensu Sun, Jieke Shi, Ting Zhang, Zilong He, Qingfu Wu, Shuai Liang, Weifeng Sun, Junda He, Pengfei Chen, Chuanfu Zhang, Lwin Khin Shar, and David Lo. AgentChaos: chaos engineering for agent systems via programmatic fault injection. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2026. arXiv:2608.06790.

[30] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.

[31] Skyler Wu, Yash Nair, and Emmanuel J. Candès. Efficient evaluation of LLM performance with statistical guarantees, 2026. arXiv:2601.20251.

[32] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: a benchmark for tool-agent-user interaction in real-world domains, 2024. arXiv:2406.12045.

[33] Shin Yoo and Mark Harman. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability*, 22(2):67–120, 2012. doi: 10.1002/stvr.430.